

Review of astar.py

Everything that is wrong, why it is wrong, what it causes, and how to fix it

How to read this. The file is a class named Astar that is meant to plan one drone at a time on a graph while respecting the moves of drones that were planned before (cooperative A*). Issues are grouped in three parts. **Part A** covers why the single-drone search is not A*. **Part B** covers why the multi-drone layer is not cooperative A*. **Part C** covers hygiene problems. Each entry shows the offending code, why it is wrong, the consequence, and the fix. Part D contains a tested reference implementation of space-time A* with a reservation table, and Part E lists what to read.

Severity. **Critical:** breaks correctness of A* or of the cooperation. **Major:** produces wrong or invalid plans in common situations. **Minor:** quality, speed or maintainability. **Design note:** a known limit of the approach rather than a bug.

The root cause in one paragraph. The code interleaves two different notions of progress. The search loop counts *expansions*, and that count is used as the *simulation time* (B1). With no accumulated g (A1), no parent links (A2), no relaxation (A3) and no (node, time) states (B2), the search cannot produce a real path, and every reservation it writes (B3, B5) is attached to a meaningless timestamp. Fixing A1-A3 and B1-B5 together is essentially a rewrite around the state (node, t); the rest are smaller corrections.

Summary of all issues

ID	Issue	Severity
A1	g is not a cost-so-far	Critical
A2	No parent pointers: parent_node is a single variable	Critical
A3	No relaxation: duplicates and unconditional overwrites	Critical
A4	Heuristic and cost model use different units (admissibility)	Major
A5	Blocked zones (infinite cost) are still reachable	Major
A6	No termination or failure handling (infinite loops)	Critical
A7	pick_node mixes node selection with constraint checking	Major
A8	Start hub is special-cased with duplicated expansion logic	Minor
A9	Non-deterministic tie-breaking	Minor
A10	Full re-sort every iteration	Minor
B1	turn counts expansions, not time	Critical
B2	Closed set contains nodes, not (node, time) states	Critical
B3	Reservations are written during the search	Critical
B4	Waiting is not a real action	Critical
B5	Only the arrival turn is reserved, not the time spent in the node	Critical
B6	Capacity is checked at the wrong turn	Major
B7	No link capacity and no swap-conflict handling	Major
B8	Restricted-zone (in-flight) modelling is inconsistent	Major
B9	Reservation table has two shapes and unused fields	Minor
B10	Return value and signature do not match	Major
B11	Start/end hub occupancy and goal behaviour are not defined	Minor

ID	Issue	Severity
B12	Sequential (prioritised) planning has no failure strategy	Design note
C1	Missing import and unused attribute	Minor
C2	Debug print in the constructor	Minor
C3	Type hints that do not match the code	Minor
C4	Dead and misleading code	Minor

Total: 26 entries (9 critical, 7 major, 9 minor, 1 design note).

Part A - Single-drone search is not A*

A1 g is not a cost-so-far

Critical

Your code

```
def get_g(self, neighbor):
    zone_score = self.get_zone_note(neighbor)
    return zone_score

def get_f(self, neighbor):
    return self.get_g(neighbor) + self.get_h(neighbor)
```

Why it is wrong. In A*, $g(n)$ is the cost of the best known path from the start to n , so $g(\text{child}) = g(\text{parent}) + \text{cost}(\text{parent}, \text{child})$. Here g is only the cost of entering one node, so $f = (\text{cost of one step}) + h$.

Consequence. The search never accounts for how far it has already travelled. It degrades into greedy best-first search: it can wander, it returns non-optimal routes, and the optimality guarantee of A* is gone. Two candidates with the same h are ranked only by their own zone cost, so a long detour looks as good as a short route.

Fix. Keep a dict $g[\text{state}]$. When expanding, compute $\text{new_g} = g[\text{current}] + \text{step_cost}$ and use $f = \text{new_g} + h$. Everything else in A* (ordering, reopening, path quality) depends on this.

A2 No parent pointers: parent_node is a single variable

Critical

Your code

```
parent_node = current_node
explored.add(current_node)
...
constraints[turn].append({"D": drone,
    "source": parent_node, "target": current_node})
```

Why it is wrong. `parent_node` holds the node that was expanded last, not the node from which `current_node` was discovered. With an open list, the next node popped can be anywhere on the frontier, not next to the last expanded node.

Consequence. Recorded moves can jump between non-adjacent nodes (drones teleport). The 'path' is not a path, the reservations describe moves that cannot happen, and the true route can never be reconstructed.

Fix. Store `came_from[child] = parent` whenever you improve a child's g . When the goal is popped, walk `came_from` backwards to rebuild the path, then reverse it.

A3 No relaxation: duplicates and unconditional overwrites

Critical

Your code

```
for neighbor in self.neighbors[current_node]:
    if neighbor in explored:
        continue
    reachable.append(neighbor)
    score[neighbor] = self.get_f(neighbor)
```

Why it is wrong. Every time a neighbor is seen it is appended again and its score is overwritten, even if the new score is worse. A* only updates a node when it finds a strictly better g (relaxation: if $\text{new_g} < g[n]$). A heap normally holds one live entry per node, or uses lazy deletion for stale ones.

Consequence. `reachable` fills with duplicates. A good score can be replaced by a worse one, so the expansion order is wrong. Stale duplicates linger in the list until they are skipped one by one, wasting time and memory.

Fix. Use `heapq`. On discovery: if $\text{new_g} < g.\text{get}(n, \text{inf})$: update g and `came_from` and push (f, tie, n) . When popping, skip entries whose state is already closed.

A4**Heuristic and cost model use different units (admissibility)****Major****Your code**

```
def get_h(self, node):
    return self.node_distances[node]      # hop count?
# zone costs: priority 0.5, normal 1, restricted 2
```

Why it is wrong. A* is optimal only if h never overestimates the real remaining cost (admissible), and ideally is consistent. If node_distances is a hop count and priority zones cost 0.5 per step, then h overestimates whenever a route uses priority zones (k priority steps cost 0.5k but h says k). In any case h and g must be in the same unit. I cannot see how node_distances is built; the argument applies whenever its units differ from g.

Consequence. A* can return a longer route than necessary. Combined with a closed set that never reopens nodes, an inadmissible h can permanently discard the better route.

Fix. Compute h with a reverse Dijkstra from the goal using exactly the same step costs as the search (the 'true distance' heuristic used by Silver's HCA*). Measure cost in turns and use priority zones as a tie-breaker instead of a fractional cost (see the reference implementation).

A5**Blocked zones (infinite cost) are still reachable****Major****Your code**

```
else:
    return float("inf")
...
reachable.append(neighbor)      # inf-cost nodes are added too
```

Why it is wrong. An infinite cost does not remove a node from the search; it only sorts it last. When every finite option is temporarily unavailable, pick_node returns the blocked node.

Consequence. Drones can be routed through zones that must be impassable.

Fix. Skip blocked zones when generating successors (step_time == inf means 'not enterable'). Also treat unknown zone strings explicitly instead of letting a typo silently become 'blocked'.

A6**No termination or failure handling (infinite loops)****Critical****Your code**

```
while self.end_hub not in explored:
    current_node = self.pick_node(...)
    ...
    while current_node is None:
        turn += 1
        current_node = self.pick_node(...)
```

Why it is wrong. The outer loop has no 'open list is empty' exit and the inner loop has no upper bound. If the goal is unreachable, reachable is empty, or every remaining node is skipped as explored, pick_node returns None forever.

Consequence. The program hangs on unreachable goals, dead ends, or heavily congested graphs, and nothing tells the caller that planning failed.

Fix. Loop on 'while open_'; return None when it empties. Add a time horizon max_t so waiting cannot go on forever, and let the caller handle a None path.

A7

pick_node mixes node selection with constraint checking

Major

Your code

```

for node in reachable:
    ...
    if node in explored: continue
    if node in occupied:
        ...
        if zone_capacity < 1: continue
    reachable.remove(node)
    return node

```

Why it is wrong. (a) It returns the first node that passes the checks at the current global turn, not the best f among nodes that are valid at their own arrival time. (b) Constraint checks belong where successors are generated, relative to the parent's time. (c) Skipped entries stay in the list forever. (d) `reachable.remove()` mutates the list being iterated (safe only because of the immediate return) and is $O(n)$.

Consequence. Best-first order is lost: a worse node is expanded whenever the best one is momentarily blocked, and the expansion order then decides timing. Explored entries and duplicates pile up.

Fix. Pop the minimum-f entry from a heap. Check reservations when creating each successor (node free at $t + \text{duration}$, link free during the move).

A8

Start hub is special-cased with duplicated expansion logic

Minor

Your code

```

if current_node == self.start_hub:
    for neighbor in self.neighbors[current_node]:
        ...
        parent_node = current_node
        explored.add(current_node)
        ...
        turn += 1
        continue

```

Why it is wrong. This block copies the general expansion code but skips the zone handling and the reservation of the node. The start is not reserved at turn 0 and its expansion consumes a turn.

Consequence. Two code paths must be kept in sync. The start hub's occupancy (if it has a finite capacity) is never counted, and every plan is shifted by an extra turn.

Fix. Treat the start as the initial state (start, 0) of the same loop. Decide explicitly whether the start hub has unlimited capacity.

A9

Non-deterministic tie-breaking

Minor

Your code

```

self.neighbors: dict[str, set[str]]
for neighbor in self.neighbors[current_node]:
    ...
    reachable = sorted(reachable, key=lambda k: score[k])

```

Why it is wrong. Iteration order of a set of strings changes from one Python process to the next (hash randomisation). `sorted()` is stable, so equal-f nodes keep whatever insertion order the set produced.

Consequence. Ties break differently on every run: different paths, different turn counts, bugs that appear and disappear.

Fix. Iterate over `sorted(neighbors)` and put an explicit tie-break in the heap key (for example prefer priority zones, then a counter).

Your code

```
reachable = sorted(reachable, key=lambda k: score[k])
```

Why it is wrong. Sorting the entire open list after each expansion costs $O(n \log n)$ per step. A binary heap costs $O(\log n)$.

Consequence. Slow on large graphs; the search is run once per drone, so the cost is multiplied by the number of drones.

Fix. Use `heapq.heappush` / `heappop`.

Part B - Multi-drone layer is not cooperative A*

B1 turn counts expansions, not time

Critical

Your code

```
turn += 1          # after every expansion (start branch, end of loop)
...
while current_node is None:
    turn += 1      # and again while 'waiting'
```

Why it is wrong. The number of nodes expanded describes search effort. The time at which a drone is at a node is the g of the path that reached it. Cooperative A* reserves (node, time) pairs, so time must belong to the state: $t_{\text{child}} = t_{\text{parent}} + \text{duration}$. Your turn is one global counter incremented for every expansion, including expansions that never end up on the drone's path.

Consequence. Reservations get timestamps the drone will never experience. Drones that would collide receive different timestamps and are not detected; other drones are blocked at times that mean nothing. This is the single biggest error in the file and it invalidates every constraint check.

Fix. Make the state (node, t). A successor's time is the parent's time plus the move's duration (wait = 1, normal = 1, restricted = 2). Never use a global counter as time.

B2 Closed set contains nodes, not (node, time) states

Critical

Your code

```
explored: set[str] = set()
...
if node in explored:
    continue
```

Why it is wrong. In space-time, (X, t=3) and (X, t=7) are different states. Marking X as closed forever prevents the drone from ever being at X at a later time.

Consequence. A drone cannot wait and come back, step into a side pocket to let another drone pass, or revisit a junction. When the only route goes through a node that was 'explored' while blocked, the search fails or hangs (A6). The algorithm loses completeness.

Fix. Closed set of (node, t) tuples. Waiting is what makes the state space finite and complete when combined with a horizon max_t.

B3 Reservations are written during the search

Critical

Your code

```
# inside the search loop, for EVERY expanded node:
constraints[turn].append({"D": drone,
    "source": parent_node, "target": current_node})
```

Why it is wrong. Reservations describe the final path. Here they are appended for every node the search expands, including dead ends and abandoned branches, directly into the shared table.

Consequence. The table fills with phantom occupancy. Later drones are blocked by ghosts (longer plans, spurious failures). If the search fails midway the garbage stays in the table.

Fix. Make the search read-only with respect to the table. After the goal is found and the path is rebuilt, call table.reserve(path) once.

B4**Waiting is not a real action****Critical****Your code**

```
while current_node is None:
    turn += 1
    current_node = self.pick_node(turn, parent_node,
                                  reachable, explored, constrains)
```

Why it is wrong. Waiting must be an explicit successor (node, t) -> (node, t+1) with its own cost and reservation. Here time advances inside a retry loop while parent_node stays fixed, and the drone then 'moves' to whichever node becomes free, possibly a different node on each retry.

Consequence. During a wait the drone has no recorded position, so other drones can plan through the tile it actually occupies. Waiting is invisible to f, and the drone's timeline has gaps.

Fix. Add (node, 1) to every node's successor list. Waits are then priced by g, reserved like any other step, and can be chosen by the search.

B5**Only the arrival turn is reserved, not the time spent in the node****Critical****Your code**

```
constrains[turn].append({"D": drone,
                        "source": parent_node, "target": current_node})
# one entry, at the arrival turn only
```

Why it is wrong. A drone occupies a node from arrival until departure. A single entry at the arrival turn blocks that instant only.

Consequence. A drone that rests at X (queued or waiting) is invisible to later drones at t+1, t+2, ... They are planned into an occupied node, which shows up as capacity violations or collisions when the plan is executed.

Fix. Represent a path as one state per turn (waits included) and reserve every (node, t). See ReservationTable.reserve in the reference implementation.

B6**Capacity is checked at the wrong turn****Major****Your code**

```
# in pick_node: built from constrains[turn] (current turn)
if node in occupied:
    zone_capacity = self.graph.zone_capacity[node] - occupied[node]

# in path_to_goal, restricted zones: arrival is turn + 1
```

Why it is wrong. For a restricted zone the drone arrives one turn later than the turn tested in pick_node, because the extra turn is added after the check.

Consequence. Restricted-zone arrivals are validated against the wrong time step: capacity can be exceeded at the real arrival time, or moves can be refused because of an unrelated turn.

Fix. Check node_free(next, t + duration) where duration already includes the restricted-zone delay.

B7**No link capacity and no swap-conflict handling****Major****Your code**

```
# if edge in occupied:
#     lint_capacity: str = self.graph.link_capacity[edge] - occupied[edge]
#     if lint_capacity < 1:
#         continue
```

Why it is wrong. Links have capacity too, and two drones exchanging positions on one link in the same turn (A to B while B to A) never share a vertex, so a vertex-only check cannot see the conflict. The edge key 'A-B' differs from 'B-A', so a naive check would also miss the reverse direction.

Consequence. Link capacity is exceeded, drones pass through each other on a link, and an invalid solution is accepted as valid.

Fix. Reserve (link, t) for every turn a drone is on a link, keyed by frozenset({a, b}) so both directions share the counter. A capacity of 1 also forbids swaps. If your rules allow one drone per direction, key by the ordered pair instead.

B8**Restricted-zone (in-flight) modelling is inconsistent****Major****Your code**

```
if self.nodes[current_node].metadata.zone == "restricted":
    edge = parent_node + "-" + current_node
    constrains[turn].append({"D": drone,
                             "source": parent_node, "target": edge})
    parent_node = edge
    turn += 1
```

Why it is wrong. (1) parent_node may be stale (A2), so the edge may not exist. (2) The edge is recorded but never capacity-checked, since pick_node only tests nodes. (3) The extra turn is added after selection, so all checks used the wrong turn (B6). (4) turn += 1 in the middle of an iteration interacts with the global counter (B1). (5) get_zone_note adds 2 to f as if it were distance, mixing cost and time.

Consequence. Ghost links in the table, arrival times off by one, and unchecked link capacity while a drone is in flight.

Fix. Model a restricted move as one successor with duration 2: the link is reserved at t and t+1, the node is reserved at t+2 only. The reference implementation does exactly this.

B9**Reservation table has two shapes and unused fields****Minor****Your code**

```
pick_node(... constrains: dict[int, dict[str, Any]])
path_to_goal(... constrains: dict[int, list[dict[str, Any]]])
# entries store "D" and "source", but nothing ever reads them
```

Why it is wrong. The same table is annotated with two different types. Each entry stores the drone id and source, but no code uses them, and every lookup scans a list.

Consequence. The table cannot answer 'is this node free at time t?' efficiently, cannot tell a drone's own reservations from those of others, and lookups are O(entries) per call.

Fix. Use counters keyed by (node, t) and (link, t), as in the reference ReservationTable.

B10**Return value and signature do not match****Major****Your code**

```
def path_to_goal(...) -> list[tuple[int, str, str]]:
    ...
    return constrains
```

Why it is wrong. The function is declared to return a list of (turn, source, target) tuples but returns the whole shared reservation table.

Consequence. The caller must scan every turn to extract one drone's moves; failure cannot be signalled; static checkers report an error.

Fix. Return this drone's path as a list of (node, t) states, or None on failure. Update the shared table separately (table.reserve(path)).

B11**Start/end hub occupancy and goal behaviour are not defined****Minor****Your code**

```
# no special handling of start_hub / end_hub in the capacity check
zone_capacity = self.graph.zone_capacity[node] - occupied[node]
```

Why it is wrong. Start and end hubs usually hold many drones at once, and a drone that reaches its goal either disappears or stays (Silver's CA* keeps agents at their goals so nobody plans through them). The code makes neither choice explicit. I cannot see how zone_capacity is filled for the hubs.

Consequence. If the hubs inherit a small capacity, drones queue or deadlock at the start or end. If finished drones should stay, later drones may plan through them.

Fix. Decide the rule explicitly. In the reference code the hubs are exempt through the unlimited set; if finished drones must stay, reserve the goal for all $t \geq \text{arrival}$.

B12**Sequential (prioritised) planning has no failure strategy****Design note****Your code**

```
# drones are planned one after another with a growing reservation table
```

Why it is wrong. This is a property of CA*, not a bug: an early drone can take the only corridor and leave a later drone with no legal plan, even though a joint solution exists. It is only sound as long as the search has a horizon and failures are handled.

Consequence. Later drones fail or wait very long, and a bad priority order gives a bad makespan.

Fix. Bound the search with max_t, detect None, and retry with a different drone order. If that is not enough, look at windowed variants (WHCA*) or Conflict-Based Search (see the reading list).

Part C - Code hygiene

C1 Missing import and unused attribute

Minor

Your code

```
self.edges: list[Connection] = self.graph.edges
```

Why it is wrong. Connection is never imported and self.edges is never used. Annotations inside a function body are not evaluated at runtime, so this does not crash, but type checkers and linters flag it.

Consequence. Noise in tooling; a real NameError appears if the annotation ever moves to class or module level or is read by typing.get_type_hints.

Fix. Import Connection under TYPE_CHECKING or remove the attribute.

C2 Debug print in the constructor

Minor

Your code

```
print(self.node_distances)
```

Why it is wrong. The constructor prints the whole distance table every time an Astar object is created.

Consequence. Noisy output, or corrupted output if your program's stdout is parsed by something else.

Fix. Remove it or use logging.debug.

C3 Type hints that do not match the code

Minor

Your code

```
def pick_node(...) -> str:          # returns None on failure
    score: dict[str, int] = {}        # get_f returns float
    parent_node: str | None          # passed to a parameter typed str
```

Why it is wrong. Several annotations are wrong: pick_node can return None, score stores floats, and parent_node is optional but the parameter is not.

Consequence. mypy/pyright errors that hide real problems, and misleading documentation.

Fix. Correct the hints (str | None, float) and run a type checker.

C4 Dead and misleading code

Minor

Your code

```
edge: str | None = None
if parent_node:
    edge = parent_node + "-" + node      # only used by commented code
# if edge in occupied: ... lint_capacity ...
```

Why it is wrong. The edge variable in pick_node is computed and never used, a commented-out block keeps the old idea alive (with a typo, lint_capacity), and the name constrains is a misspelling of constraints.

Consequence. Readers cannot tell what is intended; dead code hides that link capacity is not implemented (B7).

Fix. Delete dead code, implement or explicitly TODO the link check, and use consistent names.

Part D - Reference implementation (space-time A* with a reservation table)

This is a standalone, tested sketch. It does not depend on your Graph classes: it takes plain dicts. Adapt the names to your MakeGraph API. I tested it on small graphs (a corridor with a side pocket, a restricted zone, a swap situation, and an unreachable goal). It is not a drop-in replacement for your file, because I cannot see your Graph, Hub and metadata classes.

Assumptions you may need to change: normal and priority zones take 1 turn, restricted takes 2, blocked is not enterable; link capacity is shared by both directions; the start and end hubs are exempt from capacity (the 'unlimited' set); a drone disappears when it reaches the goal.

```
import heapq
from itertools import count

INF = float("inf")

def step_time(zone: dict[str, str], node: str) -> float:
    """Turns needed to ENTER `node`. Blocked zones are not enterable."""
    z = zone[node]
    if z == "restricted":
        return 2
    if z == "blocked":
        return INF
    return 1 # normal and priority both take 1 turn

def true_distance(neighbors, zone, goal):
    """Reverse Dijkstra from the goal, same costs as the real search.
    This is an admissible AND consistent heuristic (Silver's 'true distance')."""
    dist = {goal: 0}
    heap = [(0, goal)]
    while heap:
        d, v = heapq.heappop(heap)
        if d > dist.get(v, INF):
            continue
        for u in neighbors[v]: # graph is undirected
            nd = d + step_time(zone, v) # cost of entering v from u
            if nd < dist.get(u, INF):
                dist[u] = nd
                heapq.heappush(heap, (nd, u))
    return dist

class ReservationTable:
    def __init__(self, zone_cap, link_cap, unlimited=()):
        self.zone_cap = zone_cap # node -> capacity
        self.link_cap = link_cap # frozenset({a, b}) -> capacity
        self.unlimited = set(unlimited) # e.g. {start_hub, end_hub}
        self.node_use: dict[tuple[str, int], int] = {}
        self.link_use: dict[tuple[frozenset, int], int] = {}

    def node_free(self, node, t):
        if node in self.unlimited:
            return True
        return self.node_use.get((node, t), 0) < self.zone_cap[node]

    def link_free(self, a, b, t):
        key = frozenset((a, b))
        return self.link_use.get((key, t), 0) < self.link_cap.get(key, 1)

    def reserve(self, path):
        """path = [(node, t), ...] with one state per arrival, waits included."""
```

```

for node, t in path:
    self.node_use[(node, t)] = self.node_use.get((node, t), 0) + 1
for (a, t0), (b, t1) in zip(path, path[1:]):
    if a != b:
        # link is busy for every turn in flight
        for t in range(t0, t1):
            k = (frozenset((a, b)), t)
            self.link_use[k] = self.link_use.get(k, 0) + 1

def plan(neighbors, zone, h, table, start, goal, max_t):
    """Space-time A*. State = (node, t). Returns [(node, t), ...] or None."""
    if start not in h:
        # goal unreachable from start
        return None
    tie = count()
    s0 = (start, 0)
    g = {s0: 0}
    came_from = {}
    closed = set()
    open_ = [(h[start], 0, next(tie), s0)] # (f, non_priority_steps, tie, state)

    while open_:
        _, npri, _, state = heapq.heappop(open_)
        if state in closed:
            continue # stale duplicate
        closed.add(state)
        node, t = state

        if node == goal:
            path = [state]
            while path[-1] in came_from:
                path.append(came_from[path[-1]])
            return path[::-1]

        if t >= max_t:
            continue

        # successors: wait in place, or move along each link
        moves = [(node, 1)] + [
            (n, step_time(zone, n)) for n in sorted(neighbors[node])
            if step_time(zone, n) < INF
        ]
        for nxt, dur in moves:
            arrive = t + dur
            if h.get(nxt, INF) == INF:
                # cannot reach the goal from there
                continue
            if nxt != node and not all(
                table.link_free(node, nxt, tt) for tt in range(t, arrive)
            ):
                continue
            if not table.node_free(nxt, arrive):
                continue
            ns = (nxt, arrive)
            ng = g[state] + dur
            if ng < g.get(ns, INF):
                # relaxation
                g[ns] = ng
                came_from[ns] = state
                extra = 0 if zone[nxt] == "priority" else 1
                heapq.heappush(open_, (ng + h[nxt], npri + extra, next(tie), ns))

    return None

```

How the pieces map to your problems

A1, A2, A3, A6, A7, A10: g dict, came_from, relaxation, heap, 'while open_', return None. **A4:** true_distance() is a reverse Dijkstra with the same step costs. **A5:** step_time is infinite for blocked zones and those moves are skipped. **A9:** neighbors are iterated in sorted order and ties are broken explicitly. **B1, B2:** the state is (node, t) and the closed set holds states. **B3:** the search never writes to the table. **B4:** (node, 1) is a legal successor. **B5, B7:** reserve()

stores every (node, t) and every (link, t). **B6, B8**: a restricted move has duration 2, the link is checked at t and t+1, the node at t+2.

Driving it for many drones

```
neighbors = {n: sorted(adj) for n, adj in graph.get_graph().items()}
h = true_distance(neighbors, zone, end_hub)          # once, shared by all drones
table = ReservationTable(zone_cap, link_cap, unlimited={start_hub, end_hub})
paths = {}
for drone in range(n_drones):                       # fixed priority order
    path = plan(neighbors, zone, h, table, start_hub, end_hub, max_t=4 * len(neighbors) + n_drones)
    if path is None:
        raise RuntimeError(f"no path for drone {drone}") # or retry with another order
    table.reserve(path)                                # reserve ONCE, after the search
    paths[drone] = path
```

Suggested order of work

1. Make the single-drone search correct first (A1-A10) and compare its path length against a plain Dijkstra on random graphs. 2. Add the (node, t) state and the wait move. 3. Add the reservation table and reserve only after the search. 4. Add restricted zones and link capacity. 5. Test the classic cases: two drones swapping along a corridor, a corridor with a side pocket, a capacity-1 junction, an unreachable goal, and many drones starting at the same hub.

Part E - What to read

Cooperative pathfinding (the core references)

1. David Silver, *Cooperative Pathfinding*, AIIDE 2005, pp. 117-122. Defines CA*, HCA* and WHCA*. Free PDF: <https://cdn.aaai.org/ojs/18726/18726-52-22369-1-10-20210928.pdf> (landing page and DOI: <https://doi.org/10.1609/aiide.v1i1.18726>).
2. David Silver, *Cooperative Pathfinding*, in Steve Rabin (ed.), *AI Game Programming Wisdom 3*, Charles River Media, 2006, pp. 99-111 (ISBN 1584504579). This is the longer book chapter version. **Link status:** the URL that is commonly cited for it, http://webdocs.cs.ualberta.ca/~silver/David_Silver/Publications_files/coop-path-AIWisdom.pdf, returns a 404 error today, and I could not find a working copy of the chapter. Options: look for it through the Internet Archive (Wayback Machine) using that old URL, check the publications page on David Silver's website (davidsilver.uk), or borrow the book from a library. The 2005 paper above contains the same algorithms.

A* fundamentals

3. Red Blob Games, *Introduction to the A* Algorithm*: <https://www.redblobgames.com/pathfinding/a-star/introduction.html>. Interactive, best for understanding g, the frontier and came_from.
4. Amit Patel, *Amit's A* Pages*: <https://theory.stanford.edu/~amitp/GameProgramming/>. Heuristics, tie-breaking, implementation notes.
5. Hart, Nilsson and Raphael (1968), *A Formal Basis for the Heuristic Determination of Minimum Cost Paths*, IEEE Transactions on Systems Science and Cybernetics 4(2), 100-107. The original A* paper with the admissibility and optimality proofs.
6. Russell and Norvig, *Artificial Intelligence: A Modern Approach*, chapter on informed search (A*, admissible versus consistent heuristics, graph search versus tree search). Berkeley CS188 lectures on search cover the same material.

Multi-agent pathfinding (MAPF) in general

7. Stern et al. (2019), *Multi-Agent Pathfinding: Definitions, Variants, and Benchmarks*, Symposium on Combinatorial Search. Defines vertex, swap and capacity conflicts and maps out the field.
8. Sharon, Stern, Felner and Sturtevant (2015), *Conflict-Based Search for Optimal Multi-Agent Pathfinding*, Artificial Intelligence 219, 40-66. The standard alternative when prioritised CA* fails or gives poor results.
9. Standley (2010), *Finding Optimal Solutions to Cooperative Pathfinding Problems*, AAAI 2010. Optimal (non-prioritised) search over many agents, useful background on why the joint problem is hard.

The Silver 2005 PDF and DOI came from search results for the paper. I did not open the Red Blob Games and Amit Patel pages during this session; they are well-known references. The Alberta URL is the only link I tested directly (404).